

DATE: ___/___/___

DAY: ___/___/___

Name

Saghir Ali

Sap

67005

Assignment

04 (Algorithm)

Dept

AI - 4-1

Problem 1:-
 0/1 knapsack using
 Dynamic Programming:-

- knapsack capacity $w = 6$

item	weight	value
1	3	25
2	2	20
3	1	15
4	4	40
5	5	50

Recurrence Relation:-

$$dp(i)(w) = \begin{cases} 0 & \text{if } i=0 \text{ or } w=0 \\ dp(i-1)(w) & \text{if } w < \text{weight}[i] \\ \max(dp(i-1)(w), \text{value}[i] + dp(i-1)(w - \text{weight}[i])) & \text{if } w \geq \text{weight}[i] \end{cases}$$

DATE: ___/___/___

DAY: ___/___/___

If there are no item or capacity is 0, the value will be zero.

If the current weight is greater than the remaining capacity, the item can not be included. Otherwise we compare

- * excluding the item
- * including the item

DP Table Construction:-

Initial Table

All values are zero **Step 1**

i/w	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0

Step - 2

i/w	0	1	2	3	4	5	6
1	0	0	0	25	25	25	25

capacity less than 3 cannot include item 1.

From capacity 3 onward, value becomes 25.

Step - 3

i/w	0	1	2	3	4	5	6
2	0	0	20	25	25	45	45

DATE: ___/___/___

DAY: ___/___/___

At capacity 5, 6 both item 1 and item 2 can be selected.

Total value becomes

$$25 + 20 = 45$$

Step - 4

i/w	0	1	2	3	4	5	6
3	0	15	20	35	40	45	60

Combining item 1, 2, 3 gives

$$25 + 20 + 15 = 60$$

at capacity 6

Step - 5

i/w	0	1	2	3	4	5	6
4	0	15	20	35	40	55	60

including item 4 improves

some capacities

Maximum value at 6 still remains 60.

Step - 6

i/w	0	1	2	3	4	5	6
5	0	15	20	35	40	55	60

item 5 does not improve

final answer

Max value remains 60.

DATE: ___/___/___

DAY: ___/___/___

Final DP Table :-

W	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	25	25	25	25
2	0	0	20	25	25	45	45
3	0	15	20	35	40	45	60
4	0	15	20	35	40	55	60
5	0	15	20	35	40	55	60

Selected item

dp [5][6] = 60

selected item are

item 1

item 2

item 3

$$3+2+1 = 6$$

total

$$25+20+15 = 60$$

Find maximum value
 obtained using Dynamic
 Programming is

60

DATE: ___/___/___

DAY: ___/___/___

Problem 2 :- Algorithm Design

Pseudocode :-

Function knapSack (weight, values,
n, w)

Initialize all values as 0

For $i = 1$ to n

For $w = 0$ to w

If $\text{weights}[i-1] > w$

$\text{dp}[i][w] = \text{dp}[i-1][w]$

Else

$\text{dp}[i][w] = \max(\text{dp}[i-1][w],$

$\text{values}[i-1] + \text{dp}[i-1][w - \text{weights}[i-1]])$

Return $\text{dp}[n][w]$

C++ Program:-

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int knapSack (vector<int> & weights,  
              vector<int> & values,
```

```
              int n,
```

```
              int w)
```



```

{ vector <vector <int>> dp(n+1)
  vector <int> (w+1, 0));
for (int i=1; i<=n; i++)
{
    for (int w=0; w<=w; w++)
    {
        if (weight[i-1] > w)
        {
            dp[i][w] = dp[i-1][w];
        }
        else
        {
            dp[i][w] = max(
                dp[i-1][w],
                values[i-1]
                + dp[i-1][w - weights[i-1]]
            );
        }
    }
}
return dp[n][w];
}

```

```

int main()
{
    vector <int> weights = {7, 2, 1, 4, 5};
    vector <int> values = {25, 20, 15, 40, 30};

    int n=5;
    int w=6;

    cout << "Maximum value is"
    << knapsack(weights, values, n, w)
    << endl;
}

```


DATE: ___/___/___

DAY: ___/___/___

Problem 3:-

complexity Analysis Time :-

The algorithm uses two nested loops.

* one for item n

* one for capacities w

Therefore

$$O(n \times w)$$

For this Problem

$$5 \times 6 = 30$$

Operation performed is approximately

Space:-

The DP table stores

$$(n+1) (w+1)$$

$$\approx O(n \times w)$$

Space required is

Problem 4:- Conceptual:-

Difference b/w Greedy &
Dynamic Programming

DATE: ___/___/___

DAY: ___/___/___

Greedy

make best local choice

2/2/2024 - solve all
Sub-Problem

Faster

- More Systematic approach

Does not always give optimal answer
- Gives optimal answers

Use less memory - use extra memory

2) Why Greedy Fails for 0/1 knapsack :-

Greedy does not always give optimal answer because it can not be divided in 0/1 knapsack. It only focuses on the locally best choice.

Role of overlapping Subproblem in DP

Dynamic Programming stores result of smaller repeated problems so they are not divided again and again. This saves time and improve efficiency.